in Figure 8 the metric used by each algorithm is the KL divergence between the original trained network (with no edges patched), and the activation-patched reset network.

The reset network does not exhibit the original network's behavior, and thus it should not be possible to explain the presence of the behavior. This is a strong measure of the negation of **Q2**: if the algorithm is able to find a circuit that performs the behavior on a network that does *not* exhibit the behavior, then it will likely hallucinate circuit components in normal circumstances.

M Automated Circuit Discovery and OR gates

In this appendix we discuss an existing limitation of the three circuit discovery methods we introduced in the main text: the methods we study do not identify both inputs to 'OR gates' inside neural networks.

OR gates can arise in Neural networks from non-linearities in models. For example, if $x, y \in \{0, 1\}$ then $1 - \text{ReLU}(1 - x - y)$ is an OR gate on the two binary inputs x, y.[13] To study a toy transformer model with an OR gate, we take a 1-Layer transformer model two heads per layer, ReLU activations and model dimension 1. If both heads output 1 into the residual stream, this model implements an OR gate.[14] Our dataset (Section 2) is then equal to a single prompt where the heads output 1, and we use zero activations to test whether the circuit discovery methods can find the two inputs to the OR gate.

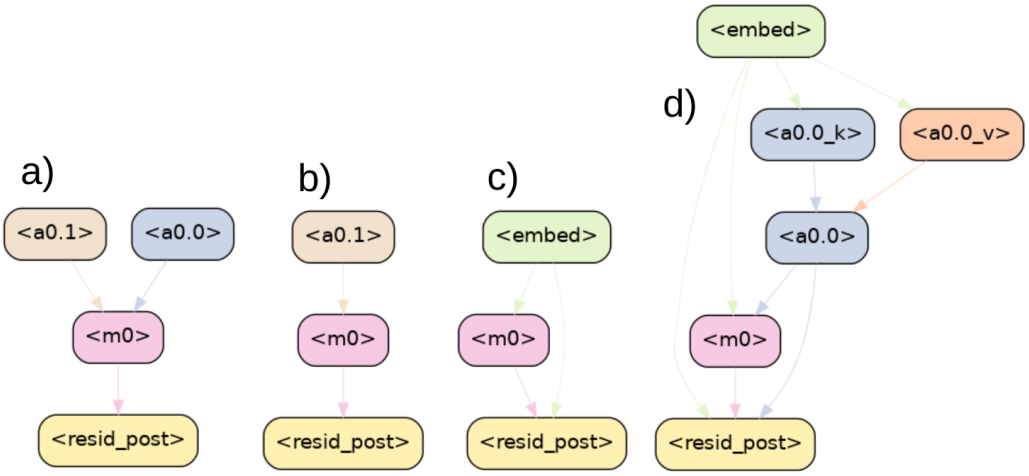

Figure 23: OR gate recovery. a) the ground truth, b) ACDC, c) HISP, d) SP.

The results can be found in Figure 23. The ground truth in a) is our toy model of an OR gate, where MLP0 performs OR on the bias terms of a0.0 and a0.1. These are the only edges that should be included. b) ACDC only recovers one OR gate input. This is because the iterative algorithm prunes the first input to the OR gate it iterates over and then keeps the other. c) HISP recovers neither OR gate input (and also recovers the unnecessary input node). d) SP recovers only one OR gate input, and several additional nodes. SP and HISP found extra edges since they include the input node by default. HISP doesn't include either attention head showing the limitations of gradients in this idealized case. We are unsure why SP finds the a0.0's key and value inputs. This shows the limitations of node-based methods for finding circuits, though ACDC is also limited. Of course, many easy fixes exist to this problem, but the priority of future work should be to explain in-the-wild language models, where it is less clear which algorithmic improvements will be most helpful. For example, follow up work found that using gradient approximations on the edges of a computational graph was very effective (Syed, Rager, and Conmy, 2023), despite not being more effective at finding OR gates.

[13] This construction is similar to the AND gate construction from Gurnee et al. (2023) Appendix A.12.

[14] For specific details of the TransformerLens (Nanda, 2022) implementation, see `https://github.com/ArthurConmy/Automatic-Circuit-Discovery/blob/main/acdc/logic_gates/utils.py#L15`.

N Connection to Causal Scrubbing

This work was inspired by work on Causal Scrubbing (Chan et al., 2022). The scope of each algorithm is quite different, however.

Causal Scrubbing (CaSc; Chan et al., 2022) aims primarily at hypothesis testing. It allows for detailed examination of specified model components, facilitating the validation of a preconceived hypothesis.

ACDC, on the other hand, operates at a broader scale by scanning over model components to generate hypotheses, each of which is tested using CaSc. ACDC chooses to remove an edge if according to the CaSc criterion, the new hypothesis isn't much worse.

Why is testing every hypothesis with Causal Scrubbing not incredibly inefficient? The reason is that ACDC only considers a small class of CaSc hypotheses, where paths through the model either matter, or don't matter. In effect, the CaSc hypotheses considered by ACDC don't allow any interchanges if the node "matters" (by having a unique value for each possible input), and the nodes that don't matter are each replaced by the same second data point.

Both methods currently face computational inefficiencies, albeit for different reasons and at different scales. Causal Scrubbing is impractical for somewhat complicated causal hypotheses because of *treeification*: there are exponentially many paths through a branching DAG, and each needs part of a forward pass. For ACDC, each hypothesis is quick to test, but the number of edges to search over can be quite large, so it still takes a while to search over circuits for realistic models. This problem is partially addressed by using gradient-based approcahes like Attribution Patching (Syed, Rager, and Conmy, 2023) or perhaps an edge-based version of Subnetwork Probing (Cao, Sanh, and Rush, 2021).

In summary, ACDC and Causal Scrubbing are complementary tools in the analysis pipeline. ACDC can do an initial coarse search over hypotheses and, while it is built on Causal Scrubbing, only considers a small class of hypotheses so it stays relatively efficient. In contrast, Causal Scrubbing offers a methodical way to test hypotheses, which can also specify the information represented in each node.